

COURSE

Основы программирования

JS

Операторы и управляющие конструкции

shesnokov@gmail.com

ОГЛАВЛЕНИЕ

План лекции ***3***

Теоретическая часть ***3***

Понятие оператора, число операндов, приоритеты операций, ассоциативность оператора **3**

Арифметические операторы **3**

Взятие остатка % 5

Инкремент/декремент: ++, -- 5

Операторы присваивания **6**

Сокращённая арифметика с присваиванием 7

Побитовые операторы **7**

Формат 32-битного целого числа со знаком 8

Список операторов 8

& (Побитовое И) 9

| (Побитовое ИЛИ) 10

^ (Исключающее ИЛИ) 10

~ (Побитовое НЕ) 10

<< (Битовый сдвиг влево) 11

>> (Правый битовый сдвиг, переносящий знак) 11

>>> (Правый сдвиг с заполнением нулями) 12

Операторы сравнения и логические значения **13**

Строгое равенство 13

Логические значения 13

Сравнение строк 13

Сравнение разных типов 14

Сравнение с null и undefined 14

Оператор запятая **15**

Взаимодействие с пользователем: alert, prompt, confirm **16**

alert 16

prompt 16

confirm 16

ВЫРАЖЕНИЯ ***16***

Основные выражения 17

Левосторонние выражения 18

Литература и другие источники ***18***

Вопросы для самопроверки ***18***

ПЛАН ЛЕКЦИИ

1. Оператор, операнд, приоритет и ассоциативность.
2. Арифметические операторы.
3. Операторы присваивания.
4. Побитовые операторы.
5. Оператор запятая.
6. Операторы сравнения.
7. Операторы взаимодействия с пользователем.
8. Выражения.

ТЕОРЕТИЧЕСКАЯ ЧАСТЬ

Понятие оператора, число операндов, приоритеты операций, ассоциативность оператора

Оператор — это элемент языка, задающий полное описание действия, которое необходимо выполнить. Каждый оператор представляет собой законченную фразу языка программирования и определяет некоторый вполне законченный этап обработки данных.

Операнд – то, к чему применяется оператор. Например, 5 * 2 – оператор умножения с левым и правым операндами. Другое название: «***аргумент оператора***».

Приоритет операторов определяет порядок, в котором операторы выполняются. Операторы с более высоким приоритетом выполняются первыми, например,

```
3 + 4 * 5 // возвращает 23
```

Оператор умножения («*») имеет более высокий приоритет, чем оператор сложения («+») и поэтому будет выполняться первым.

Ассоциативность определяет порядок, в котором обрабатываются операторы с одинаковым приоритетом. Например, рассмотрим выражение:

a OP b OP c

Левая ассоциативность (слева-направо) означает, что оно обрабатывается как (a OP b) OP c, в то время как ***правая ассоциативность (справа-налево)*** означает, что они интерпретируются как a OP (b OP c). Операторы присваивания являются право-ассоциативными, так что вы можете написать:

```
a = b = 5;
```

с ожидаемым результатом, что a и b будут равны 5. Это происходит, потому что оператор присваивания возвращает тот результат, который присваивает. Сначала b становится равным 5, затем a принимает значение b.

Арифметические операторы

Для работы с переменными, со значениями, JavaScript поддерживает все стандартные операторы, большинство которых есть и в других языках программирования.

Унарным называется оператор, который применяется к одному операнду. Например, оператор унарный минус "-" меняет знак числа на противоположный:

```
var x = 1;
x = -x;
console.log(x); // -1, применили унарный минус
```

Бинарным называется оператор, который применяется к двум операндам. Тот же минус существует и в бинарной форме:

```
var x = 1, y = 3;
console.log(y - x); // 2, бинарный минус
```

Обычно при помощи плюса '+' складывают числа. Но если бинарный оператор '+' применить к строкам, то он их объединяет в одну:

```
var a = "моя" + "строка";
console.log(a); // моястрока
```

Иначе говорят, что «плюс производит конкатенацию (сложение) строк». ***Если хотя бы один аргумент является строкой, то второй будет также преобразован к строке!***

Причем не важно, справа или слева находится операнд-строка, в любом случае не строковый аргумент будет преобразован. Например,

```
console.log('1' + 2); // "12"
console.log(2 + '1'); // "21"
```

Это приведение к строке – особенность исключительно бинарного оператора "+". Остальные арифметические операторы работают только с числами и всегда приводят аргументы к числу. Например,

```
console.log(2 - '1'); // 1
console.log(6 / '2'); // 3
```

Унарный, то есть применённый к одному значению, плюс ничего не делает с числами:

```
console.log(+1); // 1
console.log(+(1 - 2)); // -1
```

Как видно, плюс ничего не изменил в выражениях. Результат – такой же, как и без него. Тем не менее, он широко применяется, так как ***его «побочный эффект» – преобразование значения в число***. Например, когда мы получаем значения из HTML-полей или от пользователя, то они обычно в форме строк. А что, если их нужно, к примеру, сложить? Бинарный плюс сложит их как строки:

```
var apples = "2";
var oranges = "3";
console.log(apples + oranges); // "23", так как бинарный плюс складывает строки
```

Поэтому используем унарный плюс, чтобы преобразовать к числу:

```
var apples = "2";
var oranges = "3";
console.log(+apples + +oranges); // 5, число, оба операнда предварительно преобразованы в числа
```

С точки зрения математики такое изобилие плюсов может показаться странным. С точки зрения программирования – никаких разночтений: сначала выполнятся унарные плюсы, приведут строки к числам,

а затем – бинарный '+' их сложит. Почему унарные плюсы выполнились до бинарного сложения? Дело в их приоритете при выполнении операций.

Взятие остатка %

Оператор взятия остатка % интересен тем, что, несмотря на обозначение, никакого отношения к процентам не имеет. Его результат a % b – это остаток от деления a на b. Например,

```
console.log(5 % 2); // 1, остаток от деления 5 на 2
console.log(8 % 3); // 2, остаток от деления 8 на 3
console.log(6 % 3); // 0, остаток от деления 6 на 3
```

Инкремент/декремент: ++, --

Одной из наиболее частых операций в JavaScript, как и во многих других языках программирования, является увеличение или уменьшение переменной на единицу. Для этого существуют даже специальные операторы:

Инкремент ++ увеличивает на 1:

```
var i = 2;
i++; // более короткая запись для i = i + 1.
console.log(i); // 3
```

Декремент -- уменьшает на 1:

```
var i = 2;
i--; // более короткая запись для i = i - 1.
console.log(i); // 1
```

Инкремент/декремент можно применить только к переменной. Код 5++ даст ошибку. Вызывать эти операторы можно не только после, но и перед переменной: i++ (называется «постфиксная форма») или ++i («префиксная форма»). Обе эти формы записи делают одно и то же: увеличивают на 1.

Тем не менее, между ними существует разница. Она видна только в том случае, когда мы хотим не только увеличить/уменьшить переменную, но и использовать результат в том же выражении. Например,

```
var i = 1;
var a = ++i; // (*)
console.log(a); // 2
```

В строке (*) вызов ++i увеличит переменную, а затем вернёт ее значение в a. Так что в a попадёт значение i после увеличения.

Постфиксная форма i++ отличается от префиксной ++i тем, что возвращает старое значение, бывшее до увеличения.

В примере ниже в a попадёт старое значение i, равное 1:

```
var i = 1;
var a = i++; // (*)
console.log(a); // 1
```

Если результат оператора не используется, а нужно только увеличить/уменьшить переменную – без разницы, какую форму использовать:

```
var i = 0;
i++;
++i;
console.log(i); // 2
```

Если хочется тут же использовать результат, то нужна префиксная форма:

```
var i = 0;
console.log(++i); // 1
```

Если нужно увеличить, но нужно значение переменной до увеличения – постфиксная форма:

```
var i = 0;
console.log(i++); // 0
```

Инкремент/декремент можно использовать в любых выражениях. При этом он имеет более высокий приоритет и выполняется раньше, чем арифметические операции:

```
var i = 1;
console.log(2 * ++i); // 4
var i = 1;
console.log(2 * i++); // 2, выполнился раньше но значение вернул старое
console.log(i); // 2
console.log(2 * i++); // 4
console.log(i); // 3
```

При этом, нужно с осторожностью использовать такую запись, потому что в более длинной строке при быстром «вертикальном» чтении кода легко пропустить такой i++, и будет неочевидно, что переменная увеличивается. Три строки, по одному действию в каждой – длиннее, зато нагляднее:

```
var i = 1;
console.log(2 * i);
i++;
```

Операторы присваивания

Обратим внимание, в таблице приоритетов также есть оператор присваивания =. У него – один из самых низких приоритетов: 3. Именно поэтому, когда переменную чему-либо присваивают, например, x = 2 * 2 + 1 сначала выполнится арифметика, а уже затем – произойдёт присваивание =.

```
var x = 2 * 2 + 1;
console.log(x); // 5
```

Возможно присваивание по цепочке:

```
var a, b, c;
a = b = c = 2 + 2;
```

```
console.log(a); // 4
console.log(b); // 4
console.log(c); // 4
```

Такое присваивание работает справа-налево, то есть сначала вычисляется самое правое выражение 2+2, присвоится в c, затем выполнится b = c и, наконец, a = b. Рассмотрим следующий пример:

```
var a = 1;
var b = 2;
var c = 3 - (a = b + 1);
console.log(a); // 3
console.log(c); // 0
```

В примере выше результатом (a = b + 1) является значение, которое записывается в a (т.е. 3). Оно используется для вычисления c.

Сокращённая арифметика с присваиванием

Часто нужно применить оператор к переменной и сохранить результат в ней же, например,

```
var n = 2;
n = n + 5;
n = n * 2;
```

Эту запись можно укоротить при помощи совмещённых операторов, вот так:

```
var n = 2;
n += 5; // теперь n=7 (работает как n = n + 5)
n *= 2; // теперь n=14 (работает как n = n * 2)
console.log(n); // 14
```

Так можно сделать для операторов +, -, *, /, % и бинарных <<, >>, >>>, &, |, ^. Вызов с присваиванием имеет в точности такой же приоритет, как обычное присваивание, то есть выполнится после большинства других операций:

```
var n = 2;
n *= 3 + 5;
console.log(n); // 16  (n = 2 * 8)
```

Побитовые операторы

Побитовые операторы рассматривают аргументы как 32-разрядные целые числа и работают на уровне их внутреннего двоичного представления. Эти операторы не являются чем-то специфичным для JavaScript, они поддерживаются в большинстве языков программирования. Поддерживаются следующие побитовые операторы:

- AND (и) (&)
- OR (или) (|)
- XOR (побитовое исключающее или) (^)

- NOT(не) (~)
- LEFT SHIFT (левый сдвиг) (<<)
- RIGHT SHIFT (правый сдвиг) (>>)
- ZERO-FILL RIGHT SHIFT (правый сдвиг с заполнением нулями) (>>>)

Формат 32-битного целого числа со знаком

Побитовые операторы в JavaScript работают с 32-битными целыми числами в их двоичном представлении. Это представление называется «32-битное целое со знаком, старшим битом слева и дополнением до двойки». Разберём, как устроены числа внутри подробнее, это необходимо знать для битовых операций с ними.

Что такое двоичная система счисления, уже должно быть известно из курса лекций. При разборе побитовых операций будем обсуждать именно двоичное представление чисел, из 32 бит.

Старший бит слева – это научное название для самого обычного порядка записи цифр (от большего разряда к меньшему). При этом, если больший разряд отсутствует, то соответствующий бит равен нулю.

Примеры представления чисел в двоичной системе:

a = 0; // 00000000000000000000000000000000

a = 1; // 00000000000000000000000000000001

a = 2; // 00000000000000000000000000000010

a = 3; // 00000000000000000000000000000011

a = 255;// 00000000000000000000000011111111

Обратите внимание, каждое число состоит ровно из 32-битов.

Дополнение до двойки – это название способа поддержки отрицательных чисел. Двоичный вид числа, обратного данному (например, 5 и -5) получается путём обращения всех битов с прибавлением 1. То есть, нули заменяются на единицы, единицы – на нули и к числу прибавляется 1. Получается внутреннее представление того же числа, но со знаком минус. Например, вот число 314:

00000000000000000000000100111010

Чтобы получить -314, первый шаг – обратить биты числа: заменить 0 на 1, а 1 на 0:

11111111111111111111111011000101

Второй шаг – к полученному двоичному числу прибавить единицу, обычным двоичным сложением: 11111111111111111111111011000101 + 1 = 11111111111111111111111011000110.

Итак, мы получили:

-314 = 11111111111111111111111011000110

Принцип дополнения до двойки делит все двоичные представления на два множества: если крайний-левый бит равен 0 – число положительное, если 1 – число отрицательное. Поэтому этот бит называется ***знаковым битом***.

Список операторов

В следующей таблице перечислены все побитовые операторы. Далее операторы разобраны более подробно.

Оператор	Использование	Описание
Побитовое И (AND)	`a & b`	Ставит 1 на бит результата, для которого соответствующие биты операндов равны 1.
Побитовое ИЛИ (OR)	`a | b`	Ставит 1 на бит результата, для которого хотя бы один из соответствующих битов операндов равен 1.

Побитовое исключающее ИЛИ (XOR)	`a ^ b`	Ставит 1 на бит результата, для которого только один из соответствующих битов операндов равен 1 (но не оба).
Побитовое НЕ (NOT)	`~a`	Заменяет каждый бит операнда на противоположный.
Левый сдвиг	`a << b`	Сдвигает двоичное представление `a` на `b` битов влево, добавляя справа нули.
Правый сдвиг, переносящий знак	`a >> b`	Сдвигает двоичное представление `a` на `b` битов вправо, отбрасывая сдвигаемые биты.
Правый сдвиг с заполнением нулями	`a >>> b`	Сдвигает двоичное представление `a` на `b` битов вправо, отбрасывая сдвигаемые биты и добавляя нули слева.

Побитовые операторы работают следующим образом:

1. Операнды преобразуются в 32-битные целые числа, представленные последовательностью битов. Дробная часть, если она есть, отбрасывается.
2. Для бинарных операторов – каждый бит в первом операнде рассматривается вместе с соответствующим битом второго операнда: первый бит с первым, второй со вторым и т.п. Оператор применяется к каждой паре бит, давая соответствующий бит результата.
3. Получившаяся в результате последовательность бит интерпретируется как обычное число.

Например,

```
var access = parseInt("11000", 2); // получаем число из строки
console.log(access); // 24, число с таким 2-ным представлением
var access2 = access.toString(2); // обратно двоичную строку из числа
console.log(access2); // 11000
```

Без них перевод в двоичную систему и обратно был бы куда менее удобен.

& (Побитовое И)

Выполняет операцию И над каждой парой бит. Результат a & b равен единице только когда оба бита a и b равны единице. Таблица истинности для &:

a	b	a & b
0	0	0
0	1	0
1	0	0
1	1	1

Пример:

9 (по осн. 10)
= 00000000000000000000000000001001 (по осн. 2)
14 (по осн. 10)
= 00000000000000000000000000001110 (по осн. 2)
---
14 & 9 (по осн. 10)
= 00000000000000000000000000001000 (по осн. 2)
= 8 (по осн. 10)

| (Побитовое ИЛИ)

Выполняет операцию ИЛИ над каждой парой бит. Результат a | b равен 1, если хотя бы один бит из a, b равен 1. Таблица истинности для |:

a	b	a | b
0	0	0
0	1	1
1	0	1
1	1	1

Пример:

9 (по осн. 10)

= 00000000000000000000000000001001 (по осн. 2)

14 (по осн. 10)

= 00000000000000000000000000001110 (по осн. 2)

---

14 | 9 (по осн. 10)

= 00000000000000000000000000001111 (по осн. 2)

= 15 (по осн. 10)

^ (Исключающее ИЛИ)

Выполняет операцию «Исключающее ИЛИ» над каждой парой бит. a Исключающее ИЛИ b равно 1, если только a=1 или только b=1, но не оба одновременно a=b=1. Таблица истинности для исключающего ИЛИ:

a	b	a ^ b
0	0	0
0	1	1
1	0	1
1	1	0

Как видно, оно даёт 1, если ЛИБО слева 1, ЛИБО справа 1, но не одновременно. Поэтому его и называют «исключающее ИЛИ».

Пример:

9 (по осн. 10)

= 00000000000000000000000000001001 (по осн. 2)

14 (по осн. 10)

= 00000000000000000000000000001110 (по осн. 2)

---

14 ^ 9 (по осн. 10)

= 00000000000000000000000000000111 (по осн. 2)

= 7 (по осн. 10)

~ (Побитовое НЕ)

Производит операцию НЕ над каждым битом, заменяя его на обратный ему.

Таблица истинности для НЕ:

a	~a

0	1
1	0

Пример:

9 (по осн. 10)

= 00000000000000000000000000001001 (по осн. 2)

~9 (по осн. 10)

= 11111111111111111111111111110110 (по осн. 2)

= -10 (по осн. 10)

Из-за внутреннего представления отрицательных чисел получается так, что ~n == -(n+1).

Например,

```
console.log(~3); // -4
console.log(~-1); // 0
```

<< (Битовый сдвиг влево)

Операторы битового сдвига принимают два операнда. Первый – это число для сдвига, а второй – количество битов, которые нужно сдвинуть в первом операнде.

Оператор << сдвигает первый операнд на указанное число битов влево. Лишние биты отбрасываются, справа добавляются нулевые биты.

Например, 9 << 2 даст 36:

9 (по осн.10)

= 00000000000000000000000000001001 (по осн.2)

---

9 << 2 (по осн.10)

= 00000000000000000000000000100100 (по осн.2)

= 36 (по осн.10)

Операция << 2 сдвинула и отбросила два левых нулевых бита и добавила справа два новых нулевых. Левый сдвиг почти равен умножению на 2. Битовый сдвиг << N обычно имеет тот же эффект, что и умножение на два N раз, например:

```
console.log(3 << 1); // 6, умножение на 2
console.log(3 << 2); // 12, умножение на 2 два раза
console.log(3 << 3); // 24, умножение на 2 три раза
```

Конечно, следует иметь в виду, что побитовые операторы работают только с 32-битными числами, поэтому верхний порог такого «умножения» ограничен:

```
alert(10000000000 << 1); // -1474836480, отброшен крайний-левый бит
alert(10000000000 * 2); // 20000000000, обычное умножение
```

>> (Правый битовый сдвиг, переносящий знак)

Этот оператор сдвигает биты вправо, отбрасывая лишние. При этом слева добавляется копия крайнего-левого бита. Знак числа (представленный крайним-левым битом) при этом не меняется, так как

новый крайний-левый бит имеет то же значение, что и исходном числе. Поэтому он назван «переносящим знак».

Например, 9 >> 2 даст 2:

9 (по осн.10)
= 00000000000000000000000000001001 (по осн.2)
---
9 >> 2 (по осн.10)
= 00000000000000000000000000000010 (по осн.2)
= 2 (по осн.10)

Операция >> 2 сдвинула вправо и отбросила два правых бита 01 и добавила слева две копии первого бита 00.

Аналогично, -9 >> 2 даст -3:

-9 (по осн.10)
= 11111111111111111111111111110111 (по осн.2)

-9 >> 2 (по осн.10)
= 11111111111111111111111111111101 (по осн.2) = -3 (по осн.10)

Здесь операция >> 2 сдвинула вправо и отбросила два правых бита 11 и добавила слева две копии первого бита 11. , Знак числа сохранён, так как крайний-левый (знаковый) бит сохранил значение 1.

Правый сдвиг почти равен целочисленному делению на 2. Битовый сдвиг >> N обычно имеет тот же результат, что и целочисленное деление на два N раз:

```
console.log(100 >> 1); // 50, деление на 2
console.log(100 >> 2); // 25, деление на 2 два раза
console.log(100 >> 3); // 12, деление на 2 три раза, целая часть от результата
```

>>> (Правый сдвиг с заполнением нулями)

Этот оператор сдвигает биты первого операнда вправо. Лишние биты справа отбрасываются. Слева добавляются нулевые биты. Знаковый бит становится равным 0, поэтому результат всегда положителен. Для неотрицательных чисел правый сдвиг с заполнением нулями >>> и правый сдвиг с переносом знака >> дадут одинаковый результат, т.к в обоих случаях слева добавятся нули.

Для отрицательных чисел – результат работы разный. Например, -9 >>> 2 даст 1073741821, в отличие от -9 >> 2 (дает -3):

-9 (по осн.10)
= 11111111111111111111111111110111 (по осн.2)

-9 >>> 2 (по осн.10)
= 00111111111111111111111111111101 (по осн.2)
= 1073741821 (по осн.10)

Операторы сравнения и логические значения

Строгое равенство

В обычном операторе == есть «проблема» – он не может отличить 0 от false:

```
console.log(0 == false); // true
```

Та же ситуация с пустой строкой:

```
console.log('' == false); // true
```

Это естественное следствие того, что операнды разных типов преобразовались к числу. Пустая строка, как и false, при преобразовании к числу дают 0. Что же делать, если всё же нужно отличить 0 от false? Для проверки равенства без преобразования типов используются ***операторы строгого равенства*** === (тройное равно) и !==.

Если тип разный, то они всегда возвращают false:

```
console.log(0 === false); // false, т.к. типы различны
```

Строгое сравнение предпочтительно, если мы хотим быть уверены, что «сюрпризов» не будет.

Логические значения

Как и другие операторы, сравнение возвращает значение. Это значение имеет ***логический тип***. Существует всего два логических значения:

- `true` – имеет смысл «да», «верно», «истина».
- `false` – означает «нет», «неверно», «ложь».

Например,

```
console.log(2 > 1); // true, верно
console.log(2 == 1); // false, неверно
console.log(2 != 1); // true
```

Логические значения можно использовать и напрямую, присваивать переменным, работать с ними как с любыми другими:

```
var a = true; // присваивать явно
var b = 3 > 4; // или как результат сравнения
console.log(b); // false
console.log(a == b); // (true == false) неверно, выведет false
```

Сравнение строк

Строки сравниваются побуквенно:

```
console.log('Б' > 'А'); // true
```

Аналогом «алфавита» во внутреннем представлении строк служит кодировка, у каждого символа – свой номер (код). JavaScript использует кодировку Unicode. При этом сравниваются ***численные коды символов***. В частности, код у символа Б больше, чем у А, поэтому и результат сравнения такой.

В кодировке Unicode обычно код у строчной буквы больше, чем у прописной. Поэтому регистр имеет значение:

```
console.log('a' > 'Я'); // true, строчные буквы больше прописных
```

Для корректного сравнения символы должны быть в одинаковом регистре. Если строка состоит из нескольких букв, то сравнение осуществляется как в телефонной книжке или в словаре. Сначала сравниваются первые буквы, потом вторые, и так далее, пока одна не будет больше другой. Иными словами, больше – та строка, которая в телефонной книге была бы на большей странице. Например,

- Если первая буква первой строки больше – значит первая строка больше, независимо от остальных символов:

```
console.log('Банан' > 'Аят');
```

- Если одинаковы – сравнение идёт дальше. Здесь оно дойдёт до третьей буквы:

```
console.log('Вася' > 'Ваня'); // true, т.к. 'с' > 'н'
```

- Любая буква больше отсутствия буквы:

```
console.log('Привет' > 'Прив'); // true, так как 'е' больше чем "ничего".
```

Такое сравнение называется ***лексикографическим***.

Обычно мы получаем значения от посетителя в виде строк. Например, prompt возвращает *строку*, которую ввел посетитель. Числа, полученные таким образом, в виде строк сравнивать нельзя, результат будет неверен. Например,

```
console.log( "2" > "14" ); // true, неверно, ведь 2 не больше 14
```

В примере выше 2 оказалось больше 14, потому что строки сравниваются посимвольно, а первый символ '2' больше '1'. Правильно было бы преобразовать их к числу явным образом. Например, поставив перед ними +:

```
console.log( +"2" > +"14" ); // false, теперь правильно
```

Сравнение разных типов

При сравнении значений разных типов, используется числовое преобразование. Оно применяется к обоим значениям. Например,

```
console.log('2' > 1); // true, сравнивается как 2 > 1
console.log('01' == 1); // true, сравнивается как 1 == 1
console.log(false == 0); // true, false становится числом 0
console.log(true == 1); // true, так как true становится числом 1.
```

Сравнение с null и undefined

Проблемы со специальными значениями возможны, когда к переменной применяется операция сравнения > < <= >=, а у неё может быть, как численное значение, так и null/undefined. Интуитивно кажется, что null/undefined эквивалентны нулю, но это не так. Они ведут себя по-другому.

1. Значения `null` и `undefined` равны `==` друг другу и не равны чему бы то ни было ещё. Это жёсткое правило буквально прописано в спецификации языка.
2. При преобразовании в число `null` становится `0`, а `undefined` становится `NaN`.

Сравним null с нулём:

```
console.log(null > 0); // false
console.log(null == 0); // false
```

Итак, мы получили, что null не больше и не равен нулю. А теперь…

```
console.log(null >= 0); // true
```

Как такое возможно? Если нечто «больше или равно нулю», то резонно полагать, что оно либо больше, либо равно. Но здесь это не так. Дело в том, что алгоритмы проверки равенства == и сравнения >= > < <= работают по-разному.

Сравнение честно приводит к числу, получается ноль. А при проверке равенства значения null и undefined обрабатываются особым образом: они равны друг другу, но не равны чему-то ещё. В результате получается странная с точки зрения здравого смысла ситуация, которую можно увидеть в примере выше.

Значение undefined вообще нельзя сравнивать:

```
console.log(undefined > 0); // false (1)
console.log(undefined < 0); // false (2)
console.log(undefined == 0); // false (3)
```

- Сравнения (1) и (2) дают false потому, что undefined при преобразовании к числу даёт NaN. А значение NaN по стандарту устроено так, что сравнения ==, <, >, <=, >= и даже === с ним возвращают false.
- Проверка равенства (3) даёт false, потому что в стандарте явно прописано, что undefined равно лишь null и ничему другому.

Вывод: любые сравнения с undefined/null, кроме точного ===, следует делать с осторожностью. Желательно не использовать сравнения >= > < <= с ними, во избежание ошибок в коде.

Оператор запятая

Один из самых необычных операторов – запятая ','. Его можно вызвать явным образом, например,

```
var a = (5, 6);
console.log(a);
```

Запятая позволяет перечислять выражения, разделяя их запятой ','. Каждое из них – вычисляется и отбрасывается, за исключением последнего, которое возвращается. Запятая – единственный оператор, приоритет которого ниже присваивания. В выражении a = (5, 6) для явного задания приоритета использованы скобки, иначе оператор '=' выполнился бы до запятой ',', получилось бы (a=5), 6.

Зачем же нужен такой странный оператор, который отбрасывает значения всех перечисленных выражений, кроме последнего? Обычно он используется в составе более сложных конструкций, чтобы сделать несколько действий в одной строке. Например,

```
for (a = 1, b = 3, c = a*b; a < 10; a++) { // три операции в одной строке
 ...
}
```

ВЗАИМОДЕЙСТВИЕ С ПОЛЬЗОВАТЕЛЕМ: ALERT, PROMPT, CONFIRM

Данные команды не получиться использовать при отладке скрипта через Node.js. Для их работы Вы можете использовать подключение Jscript к HTML странице и ее открытие в браузере.

alert

Синтаксис: alert(сообщение). Команда alert выводит на экран окно с сообщением и приостанавливает выполнение скрипта, пока пользователь не нажмёт «ОК».

```
alert("Привет");
```

Окно сообщения, которое выводится, является ***модальным окном***. Слово «модальное» означает, что посетитель не может взаимодействовать со страницей, нажимать другие кнопки и т.п., пока не разберётся с окном. В данном случае – пока не нажмёт на «ОК».

prompt

Функция prompt принимает два аргумента:

result = prompt(title, default);

Она выводит модальное окно с заголовком title, полем для ввода текста, заполненным строкой по умолчанию default и кнопками OK/CANCEL. Пользователь должен либо что-то ввести и нажать OK, либо отменить ввод кликом на CANCEL или нажатием Esc на клавиатуре. Вызов prompt возвращает то, что ввёл посетитель – строку или специальное значение null, если ввод отменён.

Единственный браузер, который не возвращает null при отмене ввода – это Safari. При отсутствии ввода он возвращает пустую строку. Предположительно, это ошибка в браузере. Если нам важен этот браузер, то пустую строку нужно обрабатывать точно так же, как и null, т.е. считать отменой ввода. Как и в случае с alert, окно prompt модальное.

```
var years = prompt('Сколько вам лет?', 100);
alert('Вам ' + years + ' лет!')
```

Второй параметр может отсутствовать. Однако при этом IE вставит в диалог значение по умолчанию "undefined". Запустите этот код в IE, чтобы понять о чём речь:

```
var test = prompt("Тест");
```

Поэтому рекомендуется всегда указывать второй аргумент:

```
var test = prompt("Тест", ''); // <-- так лучше
```

confirm

Синтаксис: result = confirm(question); confirm выводит окно с вопросом question с двумя кнопками: OK и CANCEL. Результатом будет true при нажатии OK и false – при CANCEL(Esc). Например,

```
var isAdmin = confirm("Вы - администратор?");
alert(isAdmin);
```

ВЫРАЖЕНИЯ

Выражением является любой корректный блок кода, который возвращает значение. Концептуально, существуют два типа выражений: те, которые присваивают переменной значение, и те, которые

вычисляют значение без его присваивания. Например, выражение x = 7 - является примером выражения первого типа. Данное выражение использует оператор = для присваивания переменной x значения 7. Само выражение также равняется 7. Код 3 + 4 является примером выражения второго типа. Данное выражение использует оператор «+» для сложения чисел 3 и 4 без присваивания переменной полученного результата 7.

Все выражения в JavaScript делятся на следующие категории:

- ***Арифметические***: вычисляются в число, например, 3.14159 (используют арифметические операторы).
- ***Строковые***: вычисляются в текстовую строку, например, «Fred» или «234» (используют строковые операторы).
- ***Логические***: вычисляются в true или false (используют логические операторы).
- ***Основные выражения***: базовые ключевые слова и основные выражения в JavaScript.
- ***Левосторонние выражения***: значениям слева назначаются значения справа.

Основные выражения

Рассмотрим примеры таких выражений.

Выражения с оператором this. Используйте ключевое слово this для указания на текущий объект. В общем случае this указывает на вызываемый объект, которому принадлежит данный метод. Используйте this следующим образом:

```
this["propertyName"]
this.propertyName
```

Выражения с оператором группировки. Оператор группировки «скобки» ( ) контролирует приоритет вычисления выражений. Например, вы можете переопределить порядок - "умножение и деление, а потом сложение и вычитание", так чтобы, например, чтобы сложение выполнялось до умножения:

```
var a = 1;
var b = 2;
var c = 3;
// обычный порядок
a + b * c     // 7
// выполняется, как обычно, так
a + (b * c)   // 7
// теперь поменяем порядок -  сложение до умножения
(a + b) * c   // 9
// что эквивалентно следующему
a * c + b * c // 9
```

Упрощенный синтаксис создания массивов и генераторов. Упрощенный синтаксис - экспериментальная возможность JavaScript, которая возможно будет добавлена в будущие версии ECMAScript. Есть 2 версии синтаксиса:

[for (x of y) x] - Упрощенный синтаксис для массивов.

(for (x of y) y) - Упрощенный синтаксис для генераторов.

Упрощенные синтаксисы существуют во многих языках программирования и позволяют вам быстро собирать новый массив, основанный на существующем. Например,

```
[for (i of [ 1, 2, 3 ]) i*i ];
// [ 1, 4, 9 ]
var abc = [ "A", "B", "C" ];
[for (letters of abc) letters.toLowerCase()];
// [ "a", "b", "c" ]
```

Левосторонние выражения

Рассмотрим примеры таких выражений.

Выражения с оператором New. Вы можете использовать оператор new для создания экземпляра объекта пользовательского типа или одного из встроенных объектов. Используйте оператор new следующим образом:

```
var objectName = new objectType([param1, param2, ..., paramN]);
```

Выражения с ключевым словом super. Ключевое слово используется, чтобы вызывать функции родительского объекта. Это полезно и с классами для вызова конструктора родителя, например.

```
super([arguments]); // вызывает конструктор родителя. super.functionOnParent([arguments]);
```

Выражения с оператором расширения. Оператор расширения позволяет выражению расширяться в местах с множеством аргументов (для вызовов функций) или множестве элементов (для массивов), например,

```
var parts = ['shoulder', 'knees'];
var lyrics = ['head', ...parts, 'and', 'toes'];
```

ЛИТЕРАТУРА И ДРУГИЕ ИСТОЧНИКИ

Самый лучший общедоступный ресурс с точки зрения простоты изложения материала – это «Современный учебник по Java Script», доступный по ссылке https://learn.javascript.ru/books .

Еще один учебник Антон Кармазин «Выразительный JavaScript», доступен по ссылке https://www.gitbook.com/book/karmazzin/eloquentjavascript_ru/details .

Руководство, которое можно использовать в качестве карманного справочника - http://bonsaiden.github.io/JavaScript-Garden/ru/ .

Данных ресурсов достаточно, чтобы получить начальные навыки владения языком программирования.

Современное печатное издание, которое можно порекомендовать для новичков – Закас Н. JavaScript для профессиональных веб-разработчиков / (Пер. с англ. А. Лютича). – СПб.: Питер, 2015. – 960с.

ВОПРОСЫ ДЛЯ САМОПРОВЕРКИ

1. Чем отличается оператор от операнда?
2. На что влияет приоритет и ассоциативность операторов?
3. В чем отличие унарных, бинарных и тернарных операторов? Приведите примеры.
4. Какие операторы относятся к арифметическим?
5. В чем отличие постфиксной и префиксной записи операторов декремента и инкремента?

6. В чем отличие строгого и нестрогого присваивания?
7. Что такое «сокращенная арифметика с присваиванием»?
8. Какие операторы относятся к категории побитовых операторов?
9. Приведите примеры операторов сравнения. Особенности сравнения со специальными значениями.
10. С помощью каких операторов можно создавать диалог с пользователем?
11. Дайте определение понятию «выражение».